

Skill Week – 4

Name: N. Yeswanth Kumar

Roll No: 2520030183

Section: 8

4. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

Aim:

To design and implement parser logic in a C program that splits user input into tokens, identifies delimiters, handles whitespace, creates token structures, validates syntax, detects empty commands, and produces structured parse trees (execution structures) for a command-line interface.

Procedure:

1. **Environment Setup:** Open the WSL (Windows Subsystem for Linux) terminal to access a Linux-like command-line environment.
2. **File Creation:** Create a new C source file named Skill4.c using the nano text editor.
3. **Writing the Logic:**
 - Define Token and ParseNode structures to hold the parsed data.
 - Implement a while(1) loop to continuously prompt the user for input.
 - Add validation logic using isspace() to check for and intercept completely empty commands.

- Utilize the strtok() function to split the input string into individual tokens based on space, tab, and newline delimiters.
 - Map the first token to the Root Node (the main command) and subsequent tokens to Leaf Nodes (arguments).
4. **Compilation:** Save the file and compile the C code using the GNU C Compiler (gcc).
 5. **Execution and Testing:** Run the compiled executable and test it using standard Linux commands, programming commands, and edge cases (like multiple spaces or empty inputs) to verify the parse tree generation.

Execution:

```
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dugga\Downloads\2-1\OS\2520030106_Skill>wsl
dmvs@DMVS: /mnt/c/Users/dugga/Downloads/2-1/OS/2520030106_Skill$ nano Skill4.c
```

Code:

```
C Skill4.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <ctype.h>
5
6 // 1. Create Token Structures
7 typedef struct {
8     char text[100];
9 } Token;
10
11 // 2. Produce Execution Structures (Parse Tree Node)
12 typedef struct ParseNode {
13     char *command;
14     char *arguments[64];
15     int arg_count;
16     struct ParseNode *leftChild; // For future complex commands (like pipes |)
17     struct ParseNode *rightChild;
18 } ParseNode;
19
20 int main() {
21     char input[1024];
22     Token tokens[64]; // Validate Token Streams via array tracking
23
24     while(1) {
25         printf("parser> ");
26         if (!fgets(input, 1024, stdin)) break;
27
28         // 3. Handle Whitespace & Handle Empty Commands
29         int is_empty = 1;
30         for (int i = 0; input[i] != '\0'; i++) {
31             if (!isspace(input[i])) { is_empty = 0; break; }
32         }
33
34         if (is_empty) {
35             // Detect Errors
36             printf("[Syntax Error] Empty command detected. Please enter a valid command.\n");
37             continue;
38         }
39
40         // 4. Split Input into Tokens & Identify Delimiters (Space, Tab, Newline)
41         int token_count = 0;
42         char *token_str = strtok(input, " \n\t");
43
44         while (token_str != NULL) {
```

```

44     while (token_str != NULL) {
45         strcpy(tokens[token_count].text, token_str);
46         token_count++;
47         token_str = strtok(NULL, " \\n\\t");
48     }
49
50     // 5. Generate Parse Trees / Execution Structures
51     ParseNode root;
52     root.command = tokens[0].text;
53     root.arg_count = 0;
54     root.leftChild = NULL;
55     root.rightChild = NULL;
56
57     for (int i = 0; i < token_count; i++) {
58         root.arguments[root.arg_count++] = tokens[i].text;
59     }
60     root.arguments[root.arg_count] = NULL; // Execution structures must be NULL terminated
61
62     // 6. Debug Parsing Output
63     printf("\n--- PARSE TREE DEBUG OUTPUT ---\n");
64     printf("Root Node (Command): %s\n", root.command);
65     for (int i = 0; i < root.arg_count; i++) {
66         printf("  |-- Leaf Node (Argument %d): %s\n", i, root.arguments[i]);
67     }
68     printf("-----\n\n");
69 }
70 return 0;
71 }
72

```

```

C:\Users\dugga\Downloads\2-1\OS\2520030106_Skill>wsl
dmvs@DMVS:/mnt/c/Users/dugga/Downloads/2-1/OS/2520030106_Skill$ nano Skill4.c
dmvs@DMVS:/mnt/c/Users/dugga/Downloads/2-1/OS/2520030106_Skill$ gcc Skill4.c -o Skill4
dmvs@DMVS:/mnt/c/Users/dugga/Downloads/2-1/OS/2520030106_Skill$ ./Skill4
parser> ls -a /var/log

```

```

--- PARSE TREE DEBUG OUTPUT ---
Root Node (Command): ls
  |-- Leaf Node (Argument 0): ls
  |-- Leaf Node (Argument 1): -a
  |-- Leaf Node (Argument 2): /var/log
-----

parser> ls -l /home/user/documents

--- PARSE TREE DEBUG OUTPUT ---
Root Node (Command): ls
  |-- Leaf Node (Argument 0): ls
  |-- Leaf Node (Argument 1): -l
  |-- Leaf Node (Argument 2): /home/user/documents
-----

parser> python3 graph_traversal.py --algorithm bfs

--- PARSE TREE DEBUG OUTPUT ---
Root Node (Command): python3
  |-- Leaf Node (Argument 0): python3
  |-- Leaf Node (Argument 1): graph_traversal.py
  |-- Leaf Node (Argument 2): --algorithm
  |-- Leaf Node (Argument 3): bfs
-----

```

```

parser>
[Syntax Error] Empty command detected. Please enter a valid command.
parser> javac          MyProgram.java

--- PARSE TREE DEBUG OUTPUT ---
Root Node (Command): javac
  |-- Leaf Node (Argument 0): javac
  |-- Leaf Node (Argument 1): MyProgram.java
-----

```

Commands Used

To create and edit the file: nano Skill4.c

To compile the program: gcc Skill4.c -o Skill4

To run the executable: ./Skill4

Observation

- **Standard Parsing:** When standard commands like `ls -a /var/log` and `ls -l /home/user/documents` were entered, the program correctly identified `ls` as the Root Node (Command) and successfully separated the flags (e.g., `-a`, `-l`) and directories into distinct Leaf Nodes (Arguments).
- **Multi-Argument Parsing:** When testing a script execution command like `python3 graph_traversal.py --algorithm bfs`, the parser accurately built a tree with `python3` as the root and three distinct argument leaves.
- **Empty Command Handling:** When providing blank input (only spaces), the program successfully bypassed the parsing logic and printed a [Syntax Error] Empty command detected warning instead of crashing.
- **Delimiter Handling:** When a command was entered with excessive spaces between words (`javac MyProgram.java`), the parser correctly ignored the extra whitespace delimiters and cleanly extracted only the required tokens.

Results

The C program was successfully compiled and executed. The custom parser logic efficiently converted raw string input into structured Token and ParseNode structures. It successfully handled delimiters, caught syntax errors from empty inputs, and accurately generated terminal output representing the hierarchical Execution Structures (Parse Trees).